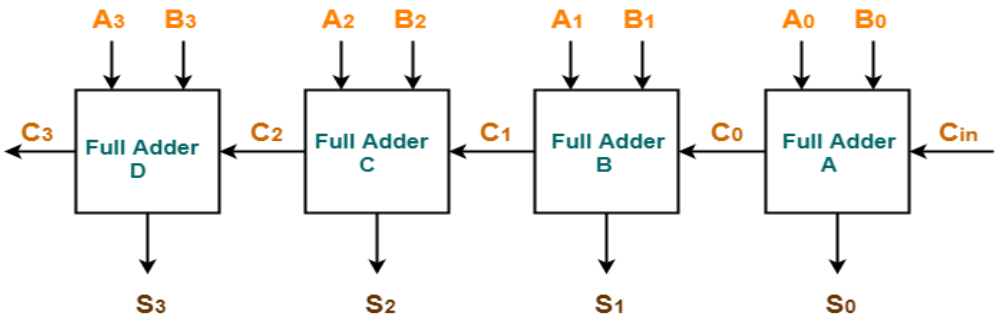
Experiment No :- 01

4-Bit Ripple Carry Adder

Aim :-

Develop and verify code, exercise a test bench synthesize and do the initial timing verification gate level simulation.

Theory :-



4-bit Ripple Carry Adder

A 4-bit Ripple Carry Adder is a digital circuit used to add two 4-bit binary numbers. It consists of four 1-bit full adders connected in series, where the carry output of one adder is passed to the carry input of the next. Each full adder adds corresponding bits from the two numbers and a carry-in. The first adder's carry-in is usually 0. The final output is a 4-bit sum and a carry-out. The term "ripple carry" refers to the way the carry signal propagates through each adder, which can cause delays.

Truth Table :-

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Verilog Code :=

```
Module fa (a,b,cin,s,c)
input a,b,cin;
output s,c;
assign s = a^b^cin;
assign c= (a&b)|(b&cin)|(cin&a);
endmodule
```

Code :-

```
Module rea(a,b,cin,s,cout)
input [3:0] a,b;
input cin;
output [3:0] s;
output cout;
wire c0, c1, c2;
fa fa0 (a[0],b[0],cin,s[0],c0);
fa fa0 (a[1],b[1],c0, s[1],c1);
fa fa0 (a[2],b[2],c1, s[2],c2);
fa fa0 (a[3],b[3],c2, s[3],c3);
endmodule
```

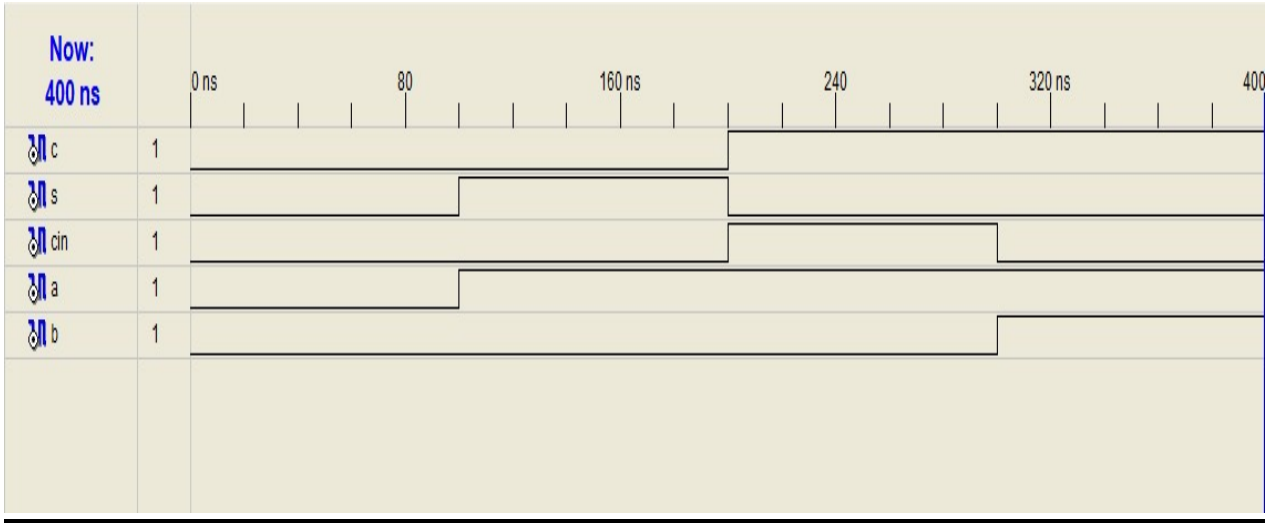
Test Bench :-

```
module rea2_v;
// Inputs
reg a;
reg b;
reg cin;
// Outputs
wire s;
```

```
wire c;
// Instantiate the Unit Under Test (UUT)
fa1 uut (
        .a(a),
        .b(b),
        .cin(cin),
        .s(s),
        .c(c)
);
initial begin
// Initialize Inputs
a = 0;
b = 0;
cin = 0;
// Wait 100 ns for global reset to finish
#100;
a = 1;
b = 0;
cin = 0;
// Wait 100 ns for global reset to finish
#100;
a = 1;
b = 0;
cin = 1;
// Wait 100 ns for global reset to finish
#100;
a = 1;
b = 1;
cin = 0;
```

```
// Wait 100 ns for global reset to finish
#100;
a = 1;
b = 1;
cin = 1;
// Wait 100 ns for global reset to finish
#100;
// Add stimulus here
end
endmodule
```

RESULT :-



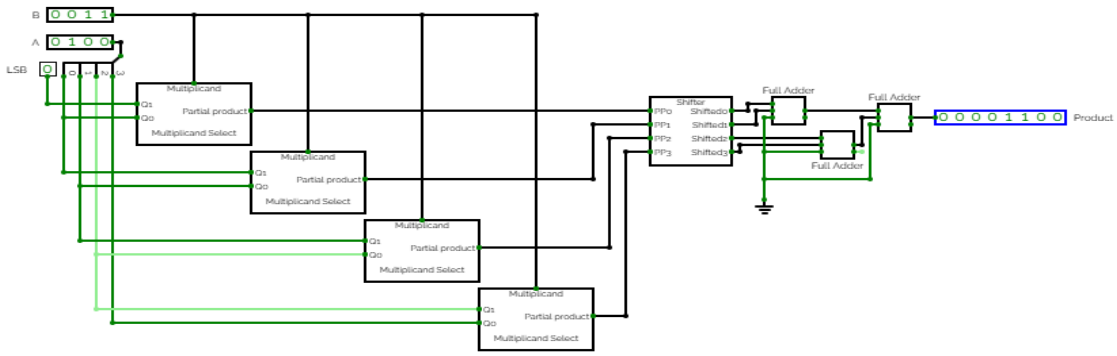
Experiment No :- 02

4 – Bit Booth Multiplier

Aim :-

Develop and verify code, exercise a test bench synthesize and do the initial timing verification gate level simulation.

Theory :-



The 4-bit Booth multiplier is a hardware algorithm used to multiply two 4-bit signed binary numbers efficiently. It applies Booth's algorithm, which reduces the number of addition and subtraction operations by encoding consecutive 1s in the multiplier. The algorithm examines pairs of bits (current and previous) to decide whether to add, subtract, or do nothing with the multiplicand. Shifting and arithmetic operations are used to accumulate the partial products. This method improves performance, especially for signed and repetitive-bit numbers.

Truth Table :-

b_{i+1}	b_i	b_{i-1}	value	*X1_a*	*X2_b*	*Z*	*Neg*
0	0	0	0	1	0	1	0
0	0	1	1	0	1	1	0
0	1	0	1	0	1	0	0
0	1	1	2	1	0	0	0
1	0	0	-2	1	0	0	1
1	0	1	-1	0	1	0	1
1	1	0	-1	0	1	1	1
1	1	1	0	1	0	1	1

Verilog Code :-

```
Module boothmulti (X,Y,Z);
input signed [3:0] X,Y;
output signed [7:0] Z;
reg signed [7:0] Z;
reg [1:0] temp;
integer i;
reg E1;
reg [3:0] Y1;
always @(X,Y)
begin
Z=8'd0;
E1=1'd0;
for (i=0; i<4; i=i+1)
begin
temp={X[i], E1};
Y1 = -Y;
case(temp)
```

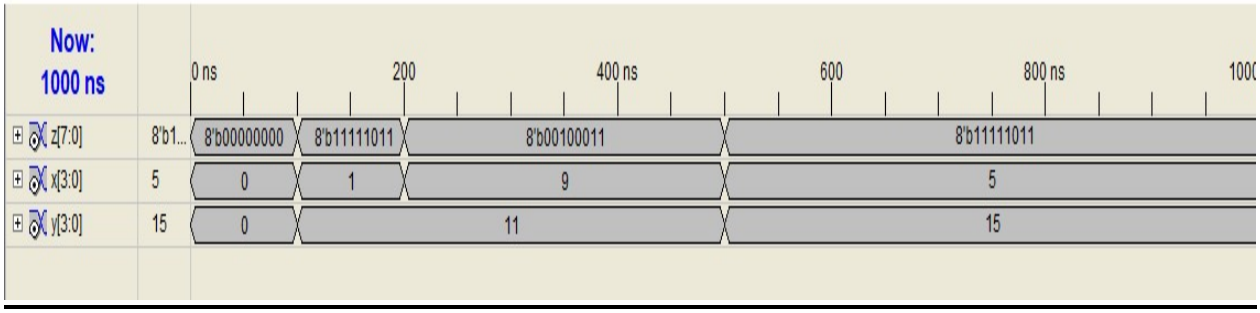
```
2'd2: Z[7:4]=Z[7:4]+Y1;
2'd1: Z[7:4]=Z[7:4]+Y;
default: begin end
endcase
Z=Z>>1;
Z[7]=Z[6];
E1=X[i];
end
if(Y==4'd8)
begin
Z= -Z;
end
end
end module
```

Test Bench :-

```
module boo1_v;
reg[3:0] X;
reg[3:0] Y;
wire [7:0] Z;
boo uut(
   .X(X),
   .Y(Y),
   .Z(Z)
);
initial begin
X=0000;
Y=0000;
```

```
#100;
X=0001;
Y=0011;
#100;
X=1001;
Y=0011;
#100;
X=1001;
Y=1011;
#100;
X=0101;
Y=1010;
#100;
end
end module
```

Result :-



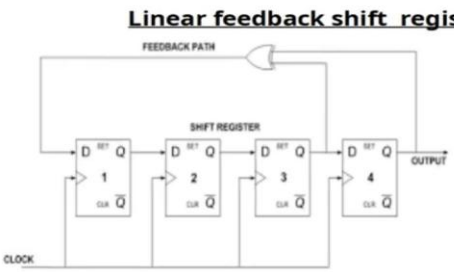
Experiment No :-03

a) LFSR

Aim :-

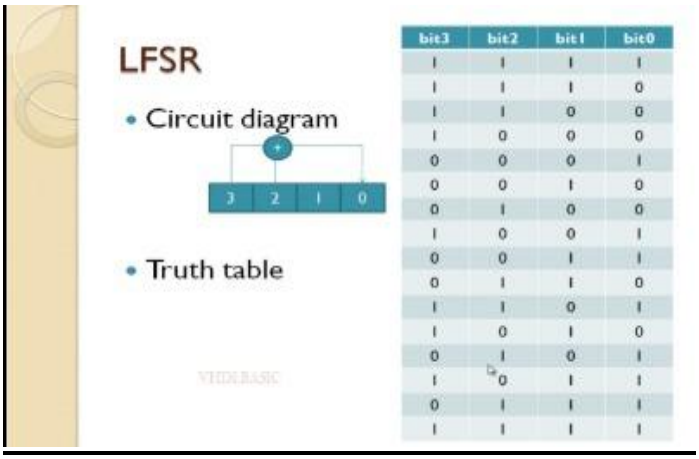
Develop and verify code, exercise a test bench synthesize and do the initial timing verification gate level simulation.

Theory :-



A Linear Feedback Shift Register (LFSR) is a sequential shift register used to generate pseudo-random binary sequences. It consists of a series of flip-flops connected in a line, with some outputs fed back through XOR gates to the input. The feedback taps are determined by a characteristic polynomial. At each clock pulse, the bits shift right, and the new bit is a linear function (XOR) of selected previous bits. LFSRs are widely used in cryptography, error detection, and digital signal processing due to their simplicity and efficiency.

Truth Table :-



bit3	bit2	bit1	bit0
1	1	1	1
1	1	1	0
1	1	0	0
1	0	0	0
0	0	0	1
0	0	1	0
0	1	0	0
1	0	0	1
0	0	1	1
0	1	1	0
1	1	0	1
1	0	1	0
0	1	0	1
1	0	1	1
0	1	1	1
1	1	1	1

Verilog Code :-

```
module ytqwrrt(clk,rst, out);
input clk,rst;
output reg [3:0] out;
always@(posedge clk,posedge rst)
if(rst)
out<=4'hf;
else
out<={out[2:0],(out[3]^out[2])};
endmodule
```

Test Bench :-

```
module sfytte_v;
// Inputs
reg clk;
reg rst;
// Outputs
wire [3:0] out;
// Instantiate the Unit Under Test (UUT)
ytqwrrt uut (
        .clk(clk),
        .rst(rst),
        .out(out)
);
initial begin
// Initialize Inputs
clk = 0;
rst = 1;
```

#50;

rst = 0;

#50;

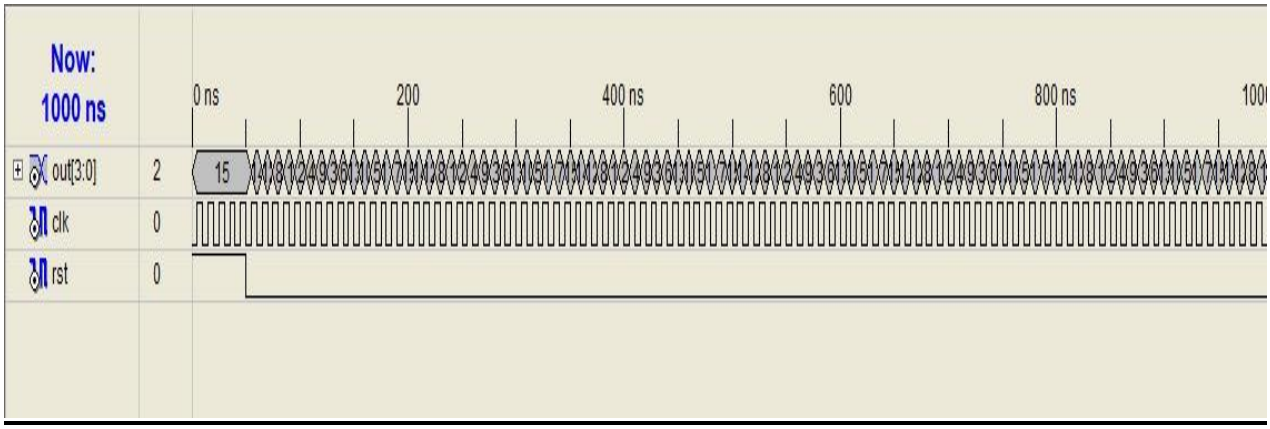
// Add stimulus here

end

always#5 clk=~clk;

endmodule

Output :-

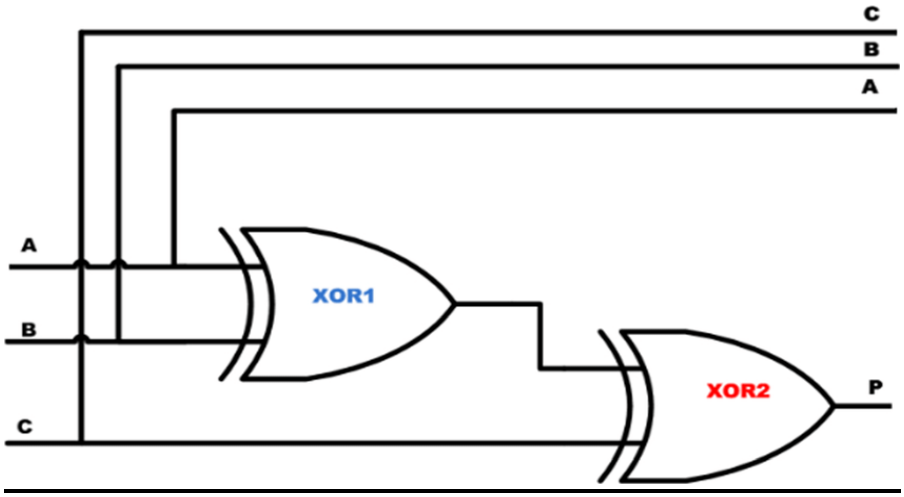


b) Parity Generators :-

Aim :-

Develop and verify code, exercise a test bench synthesize and do the initial timing verification gate level simulation.

Theory :-



A parity generator is a digital circuit used to create a parity bit for error detection in data transmission. It adds an extra bit (parity bit) to a binary word to ensure the total number of 1s is either even (even parity) or odd (odd parity). The circuit typically uses XOR gates to calculate the parity bit based on the input data bits. Even parity outputs 1 if the number of 1s is odd, and 0 if it's even (and vice versa for odd parity). Parity generators are commonly used in communication systems and memory storage for basic error checking.

Truth Table :-

A	B	C	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Verilog Code :-

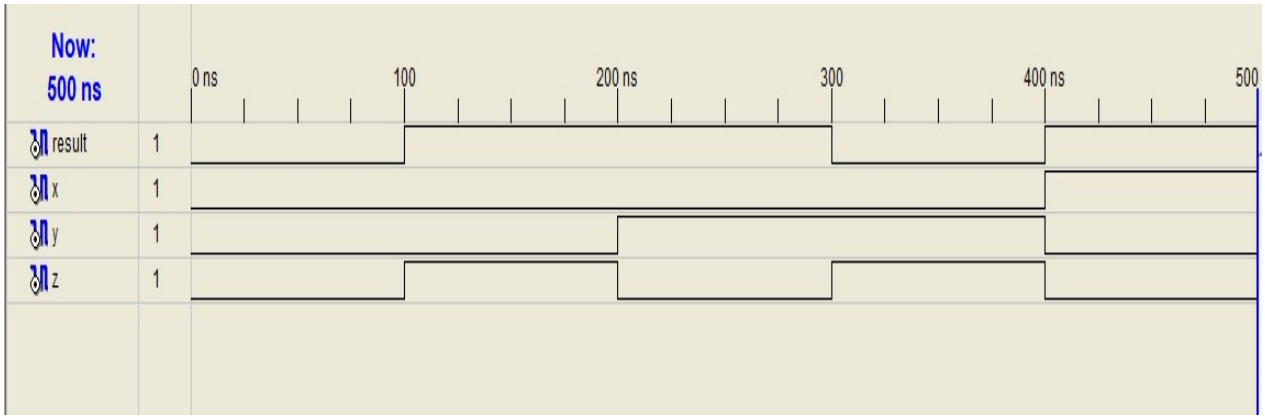
```
module wgdyu(x,y,z, result);
input x,y,z;
output result;
xor(result,x,y,z);
endmodule
```

Test Bench :-

```
module rsrst_v;
// Inputs
reg x;
reg y;
reg z;
// Outputs
wire result;
// Instantiate the Unit Under Test (UUT)
wgdyu uut (
        .x(x),
        .y(y),
        .z(z),
        .result(result)
);
initial begin
// Initialize Inputs
x = 0;
y = 0;
z = 0;
// Wait 100 ns for global reset to finish
```

```
#100;
x = 0;
y = 0;
z = 1;
#100;
x = 0;
y = 1;
z = 0;
#100;
x = 0;
y = 1;
z = 1;
#100;
x = 1;
y = 0;
z = 0;
#100;
x = 1;
y = 1;
z = 1;
#100;
// Add stimulus here
end
endmodule
```

Result :-



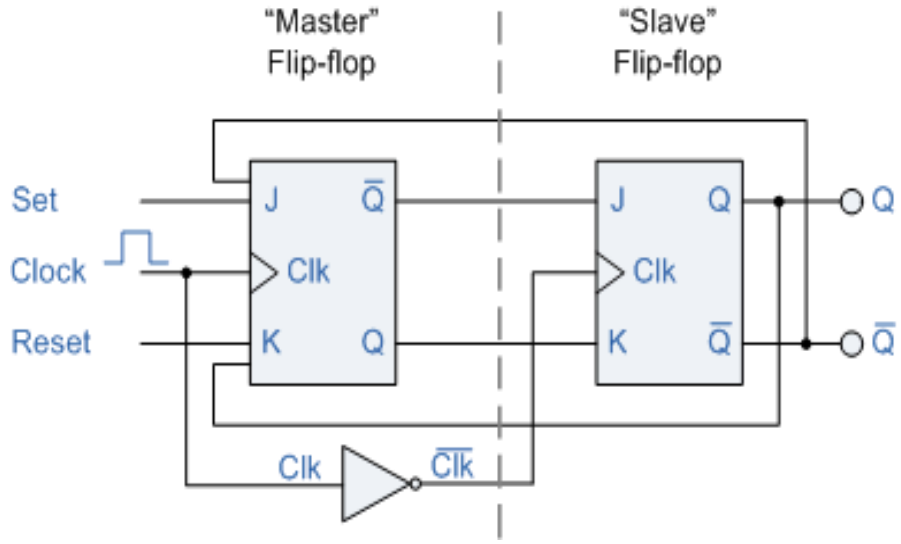
Experiment No :- 04

Master Slave JK Flip Flop

Aim :-

Develop and verify code, exercise a test bench synthesize and do the initial timing verification gate level simulation.

Theory :-



A Master-Slave JK Flip-Flop is a sequential logic circuit that eliminates the race-around condition found in basic JK flip-flops. It consists of two JK flip-flops connected in series: the "master" and the "slave." The master is triggered by the clock's rising edge, while the slave is triggered by the falling edge. This arrangement ensures that the output changes only once per clock cycle, providing stable and predictable behavior. The flip-flop responds to inputs J and K similarly to the SR flip-flop but allows toggling when both inputs are high. It is widely used in counters, registers, and memory devices.

Truth Table :-

Clock	J	K	Q_n	Master flip-flop output	Slave flip-flop output (Q_{n+1})
↑	0	0	0	0	No Change
↓	0	0	0	No Change	0
↑	0	0	1	1	No Change
↓	0	0	1	No Change	1
↑	0	0	0	0	No Change
↓	0	0	0	No Change	0
↑	0	0	1	0	No Change
↓	0	0	1	No Change	0
↑	1	1	0	1	No Change
↓	1	1	0	No Change	1
↑	1	1	1	1	No Change
↓	1	1	1	No Change	1
↑	1	1	0	1	No Change
↓	1	1	0	No Change	1
↑	1	1	1	0	No Change
↓	1	1	1	No Change	0

Verilog Code :-

```
module msjk (j,k,clk,qm,qmb,qs,qsb);
input j,k,clk;
output qm,qmb,qs,qsb;
reg qm,qmb,qs,qsb;
always @(posedge clk)
begin
if (j==0 & k==1)
qm=0;
else if (j==1 & k==0)
qm=1;
else if (j==0 & k==0)
qm= qm;
else if (j==1 & k==1)
qm =~qm;
end
```

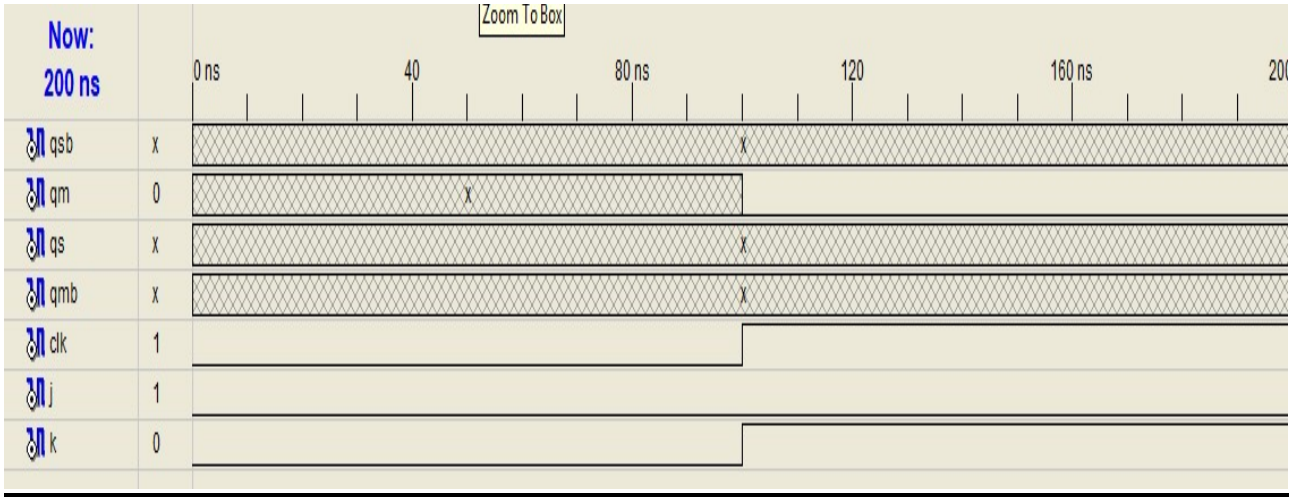
```
always @(negedge clk)
begin
qs=qm;
qsb=~qs;
end
endmodule
```

Test Bench :-

```
module rsrst_v;
// Inputs
reg J;
reg K;
reg qm;
reg qmb;
reg qs;
req qsb;
// Outputs
wire result;
// Instantiate the Unit Under Test (UUT)
wgdyu uut (
        .J(J),
        .K(K),
        .qm(qm),
        .qmb(qmb),
        .qs(qs),
        .qsb(qsb)
);
initial begin
```

```
always #10 clk=~clk;
J=0;
K=0;
Clk=0;
#100;
J=0;
K=1;
Clk=1;
#100;
J=1;
K=0;
#100;
end
endmodule
```

Result :-



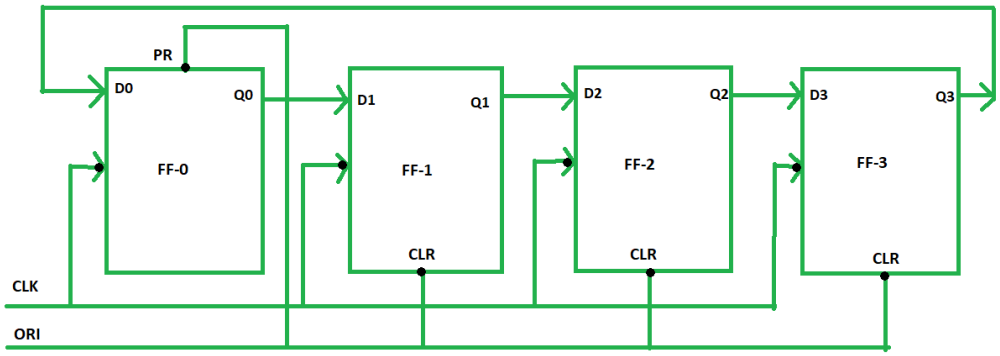
Experiment No :- 05

4 – Bit Ring Counter

Aim :-

Develop and verify code, exercise a test bench synthesize and do the initial timing verification gate level simulation.

Theory :-



Straight Ring Counter

A 4-bit Ring Counter is a type of sequential circuit made up of four flip-flops connected in a circular fashion. It starts with one flip-flop set to '1' and the others to '0'. With each clock pulse, the '1' bit shifts to the next flip-flop in the sequence. After four clock cycles, the pattern repeats, forming a ring. It is commonly used for timing, sequencing, and controlling operations in digital systems. Ring counters are simple and require no decoding logic.

Truth Table :-

PRESETED 1

ORI	CLK	Q0	Q1	Q2	Q3
low	X	1	0	0	0
high	low	0	1	0	0
high	low	0	0	1	0
high	low	0	0	0	1
high	low	1	0	0	0

Verilog Code :-

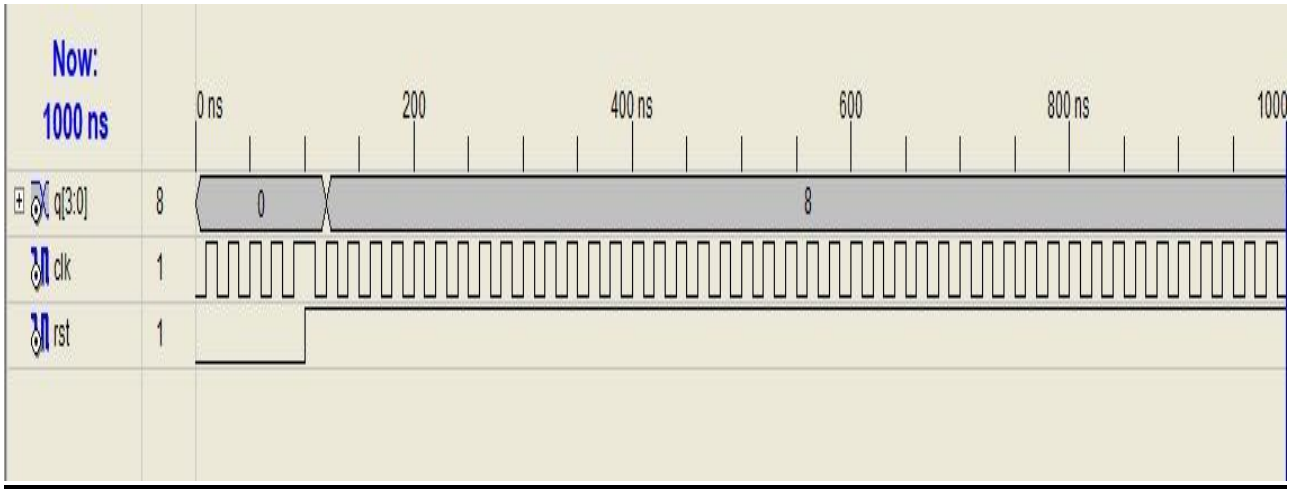
```
module jgduf(clk,rst, q);
input clk,rst;
output reg [3:0] q=4'b0000;
always@(posedge clk)
begin
if(rst==1)
q<=4'b1000;
else
begin
q[3]<=q[0];
q[2]<=q[3];
q[1]<=q[2];
q[0]<=q[1];
end
end
endmodule
```

Test Bench :-

```
module sdfgh_v;
// Inputs
reg clk;
reg rst;
// Outputs
wire [3:0] q;
// Instantiate the Unit Under Test (UUT)
gfds uut (
        .clk(clk),
```

```
        .rst(rst),
        .q(q)
);
always#10 clk=~clk;
initial begin
// Initialize Inputs
clk = 0;
rst = 0;
// Wait 100 ns for global reset to finish
#100;
clk = 1;
rst = 1;
// Wait 100 ns for global reset to finish
#100;
// Add stimulus here
end
endmodule
```

Result :-



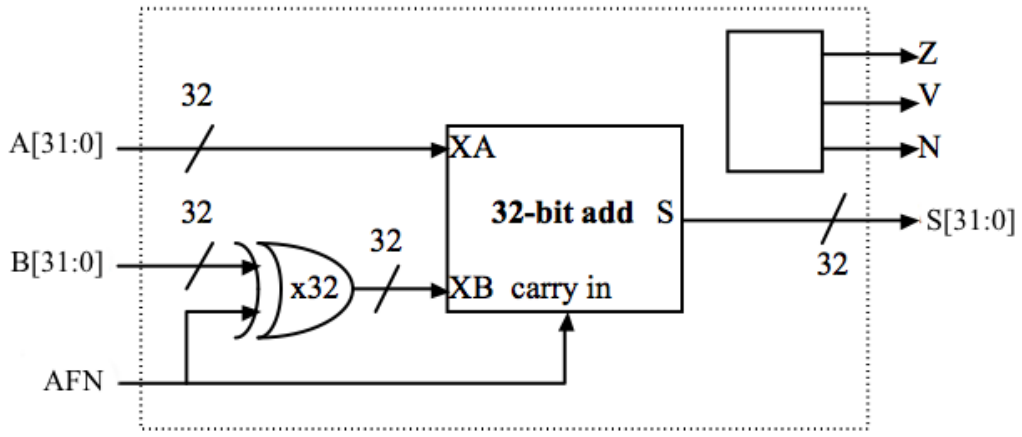
Experiment No :- 06

32 – Bit ALU Supporting 4-Logical and 4- Arithmetic operations

Aim :-

Develop and verify code, exercise a test bench synthesize and do the initial timing verification gate level simulation.

Theory :-



A 32-bit ALU (Arithmetic Logic Unit) is a digital circuit that performs arithmetic and logical operations on 32-bit binary numbers. It supports 4 arithmetic operations such as addition, subtraction, increment, and decrement, and 4 logical operations like AND, OR, XOR, and NOT. The ALU takes two 32-bit inputs and a control signal to select the desired operation. It produces a 32-bit result along with status flags (e.g., zero, carry, overflow). This unit is essential in CPUs for executing mathematical and logical instructions efficiently.

Truth Table :-

INPUT 1	INPUT 2	INPUT 3	OPERATIONS
0	0	0	~A
0	0	1	A|B
0	1	0	A&B
0	1	1	-A
1	0	0	A+B
1	0	1	A-B
1	1	0	A*B
1	1	1	(B!=0)?A/B:0

Verilog Code :-

```
module ALU-32 bit ( input [31:0] A,B, input[2:0] ALU_sel, output reg [31:0] ALU_out);
always @(*) begin
case (ALU_sel)
3'b000 ALU_out = ~A;
3'b001 ALU_out = A|B;
3'b010 ALU_out = A&B;
3'b011 ALU_out = -A;
3'b100 ALU_out = A+B;
3'b101 ALU_out = A – B;
3'b110 ALU_out = A*B;
3'b111 ALU_out = (B!=0)?A/B:0;
default : ALU_out = 32'b0;
endcase
end
end module
```

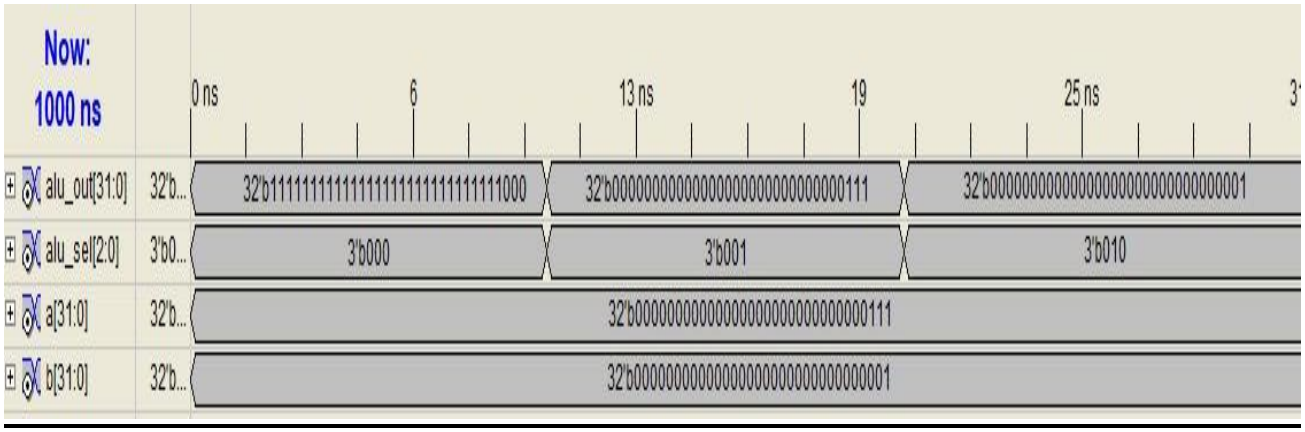
Test Bench :-

```
module dfg_v;
// Inputs
reg [31:0] a;
reg [31:0] b;
reg [2:0] alu_sel;
// Outputs
wire [31:0] alu_out;
// Instantiate the Unit Under Test (UUT)
sdsf uut (
```

```
        .a(a),
        .b(b),
        .alu_sel(alu_sel),
        .alu_out(alu_out)
);
initial begin
// Initialize Inputs
a = 32'h00000007;
b =32'h00000001 ;
alu_sel =3'b000;
// Wait 100 ns for global reset to finish
#10;
alu_sel =3'b001;
#10;
alu_sel =3'b010;
#50;
// Add stimulus here

end
endmodule
```

Result :-


Now:
1000 ns
0 ns
6
13 ns
19
25 ns
alu_out[31:0]
32'b...
32'b11111111111111111111111111111000
32'b00000000000000000000000000000111
32'b00000000000000000000000000000001
alu_sel[2:0]
3'b0...
3'b000
3'b001
3'b010
a[31:0]
32'b...
32'b00000000000000000000000000000111
b[31:0]
32'b...
32'b00000000000000000000000000000001